

Introduction to Social Data Analytics: Week 6 Part 1

Discrimination in the Labor Market

- ▶ Pervasive evidence of inequality in the labor market
 - ▶ Black workers are about twice as likely as white workers to be unemployed (Council of Economic Advisors, 1998)
- ▶ However, often difficult to isolate discrimination from employers as source of a given disparity
 - ▶ Some argue researchers do not have all factors that go into an employer's decision to hire a worker, therefore differences in hiring rates across different races could be due to these other factors

Bertrand and Mullainathan (2004)

- ▶ To isolate discrimination, Bertrand and Mullainathan (2004) send fictitious resumes to help-wanted ads in Boston and Chicago
- ▶ They manipulate the perception of race by altering the name on the resume
 - ▶ Examples of white-sounding names from the paper: Emily Walsh and Greg Baker
 - ▶ Examples of Black-sounding names from the paper: Lakisha Washington and Jamal Jones
- ▶ All other characteristics of the applicant (experience, education, job history) are randomized
- ▶ They then compare callback rates (for an interview) between Black and white applicants

Resume Audit Study

- ▶ This type of empirical design is known as a resume audit study or a correspondence study
- ▶ Key strength of the design: all other factors relevant for a job are held fixed
- ▶ Therefore disparities between races cannot be “explained away” by other factors
 - ▶ All factors are observed and similar between white and Black applicants
- ▶ Before getting to the data though, we need to learn how to use R!

RStudio Interface

The screenshot displays the RStudio IDE interface. The top menu bar includes File, Edit, Code, View, Plots, Session, Build, Debug, Tools, and Help. The toolbar below the menu contains icons for file operations, running code, and source control. The main Editor window shows a script with three lines of R code: `1 a <- 25`, `2 b <- 3`, and `3 a*b`. The Environment pane on the right shows the Global Environment with variables `a` (value 25) and `b` (value 3). The Console window at the bottom left shows the execution of the code: `> a <- 25`, `> b <- 3`, `> a*b`, resulting in `[1] 75`. The Help pane on the bottom right shows the documentation for the `read.table` function, including its description and usage.

Editor window

```
1 a <- 25
2 b <- 3
3 a*b
```

Environment

Values	
a	25
b	3

Console window

```
~/Documents/Uni/Projects/qetxrbok/ Working directory
> a <- 25
> b <- 3
> a*b
[1] 75
> |
```

Help / Packages / Plots / Files

Data Input

Description

Reads a file in table format and creates a data frame from it, with cases corresponding to lines and variables to fields in the file.

Usage

```
read.table(file, header = FALSE, sep = "", quote = "\"",
  dec = ".", numerals = c("allow.loss", "warn.loss", "no.loss"),
  row.names, col.names, as.is = !stringsAsFactors,
  na.strings = "NA", colClasses = NA, nrows = -1,
  skip = 0, check.names = TRUE, fill = !blank.lines.skip,
  strip.white = FALSE, blank.lines.skip = TRUE,
  comment.char = "#",
```

Arithmetic Operations

R can be used as a calculator:

```
5 + 3
```

```
## [1] 8
```

```
5 - 3
```

```
## [1] 2
```

```
5*3
```

```
## [1] 15
```

```
5/3
```

```
## [1] 1.666667
```

```
5^3
```

```
## [1] 125
```

Order of operations

Parentheses can be used to specify order of operations

```
(5+3)/3
```

```
## [1] 2.666667
```

```
5 + 3/2
```

```
## [1] 6.5
```

Functions are used for more complication mathematical computations

```
log(5)
```

```
## [1] 1.609438
```

```
sqrt(2)
```

```
## [1] 1.414214
```

```
abs(-1)
```

```
## [1] 1
```

The question mark can be used to pull up help files

```
?log  
?sqrt  
?abs
```

Hashtags create comments that are not run

```
#Calculate the hypotenuse of a triangle  
#One side = 5  
#The other side = 4  
#The hypotenuse is:  
sqrt(52 + 42)
```

```
## [1] 6.403124
```

R Objects

Objects, any pieces of information stored by R, can be:

- ▶ A number, called “numeric” in R (e.g. 5.23)
- ▶ A text string, called “character” in R (e.g. “UCSD is awesome”)
- ▶ A dataset
- ▶ Many more!

Creating objects

R can store *objects* with a name of our choice. Use `<-` or `=` as an assignment operator for objects.

```
graduationdate <- 2010  
graduationdate = 2010  
graduationdate
```

```
## [1] 2010
```

We can then use our new object in calculations

```
#Calculate years since graduation  
2022 - graduationdate
```

```
## [1] 12
```

Creating objects (cont.)

If we assign a new value to the same object name, then we will overwrite this object (so be careful when doing so!)

```
graduationdate <- 2014  
graduationdate
```

```
## [1] 2014
```

Objects (cont.)

R can also represent other types of values as objects, such as strings of characters:

```
school <- "UCSD"
```

We can use the class function to retrieve the type of object

```
class(graduationdate)
```

```
## [1] "numeric"
```

```
class(school)
```

```
## [1] "character"
```

A *vector* represents a collection of information of the same type stored in a specific order

We use the function `c()`, which stands for “concatenate,” to enter a data vector (with commas separating elements of the vector):

```
#Create a vector of graduation dates  
graduationdate <- c(2010, 2009, 2015, 2001)  
graduationdate
```

```
## [1] 2010 2009 2015 2001
```

We can then perform calculations on vectors

```
#Calculate years since graduation  
2022 - graduationdate
```

```
## [1] 12 13 7 21
```

Vectors (cont.)

We can create a character vector

```
#Create a vector of schools  
school <- c("UCSD", "UCB", "UCLA", "UCR")
```

Notice that when we combine a character and numeric vector, R changes the whole thing to a character vector

```
c(graduationdate, school)
```

```
## [1] "2010" "2009" "2015" "2001" "UCSD" "UCB"  "UCLA" "UCR"
```

Vectors (cont.)

We also add numeric vectors together

```
#Create a vector of income  
income <- c(65, 70, 20, 100)  
  
#Create a vector of raises  
raise <- c(5, 10, 3, 20)  
  
#Calculate current income for each person  
income + raise  
  
## [1] 70 80 23 120
```

Functions

A *function* takes input object(s) and returns an output object. In R, a function generally runs as `funcname(input)`. Some basic functions useful for summarizing numeric vectors include:

- ▶ `length()`: length of a vector (number of elements)
- ▶ `min()`: minimum value
- ▶ `max()`: maximum value
- ▶ `range()`: range of data
- ▶ `mean()`: mean
- ▶ `sum()`: sum

We can try these with our vectors.

Functions (cont.)

```
#Maximum income
```

```
max(income)
```

```
## [1] 100
```

```
#Minimum graduate date
```

```
min(graduationdate)
```

```
## [1] 2001
```

```
#Length of the vector income
```

```
length(income)
```

```
## [1] 4
```

Functions (cont.)

```
#Range of graduation dates  
range(graduationdate)
```

```
## [1] 2001 2015
```

```
#Mean years since graduation  
mean(2022 - graduationdate)
```

```
## [1] 13.25
```

```
#Sum of income  
sum(income)
```

```
## [1] 255
```

Functions (cont.)

Notice that many of these functions require a numeric vector

```
#Mean of school is undefined
```

```
mean(school)
```

```
## Warning in mean.default(school): argument is not numeric or logical: re
```

```
## NA
```

```
## [1] NA
```

Finding your working directory in R

- ▶ Go to Session → Set Working Directory → Choose Directory
- ▶ Select the folder that you have the resume data in.
- ▶ The code with the address to your working directory will appear in your console
- ▶ Save that code to your .R file.

Using code to specify your working directory

- ▶ Use the function `setwd()` to change the working directory

```
setwd()
```

- ▶ Use the function `getwd()` to display the current working directory.

```
getwd()
```

Loading data from your working directory

- ▶ For CSV files:

```
resume <- read.csv("resume.csv")
```

- ▶ For *RData* files:

```
resume <- load("resume.RData")
```

Data Frames

A *data frame* is a collection of vectors, but we can think of it like an Excel spreadsheet. Useful functions for data frames include:

- ▶ `names()`: return a vector of variable names
- ▶ `nrow()`: return the number of rows
- ▶ `ncol()`: return the number of columns
- ▶ `dim()`: combine `ncol()` and `nrow()` into a vector
- ▶ `summary()`: produce a summary
- ▶ `head()`: displays the first six observations
- ▶ `tail()`: displays the last six observations

Load `resume.csv`, assign it to an object called `resume`, and try the above functions on this newly created data frame.

Data Frames (cont.)

```
names(resume)
```

```
## [1] "X"          "firstname" "sex"        "race"       "call"
```

```
nrow(resume)
```

```
## [1] 4870
```

```
ncol(resume)
```

```
## [1] 5
```

Data Frames (cont.)

```
dim(resume)
```

```
## [1] 4870    5
```

```
summary(resume)
```

```
##           X           firstname           sex           race
##  Min.      :    1  Length:4870      Length:4870      Length:4870
## 1st Qu.:1218  Class :character  Class :character  Class :character
## Median :2436  Mode  :character  Mode  :character  Mode  :character
## Mean      :2436
## 3rd Qu.:3653
## Max.      :4870
##           call
##  Min.      :0.00000
## 1st Qu.:0.00000
## Median :0.00000
## Mean      :0.08049
```

Data Frames (cont.)

```
head(resume)
```

```
##      X firstname      sex  race call
## 1 1    Allison female white     0
## 2 2    Kristen female white     0
## 3 3    Lakisha female black     0
## 4 4    Latonya female black     0
## 5 5     Carrie female white     0
## 6 6         Jay   male white     0
```

```
tail(resume)
```

```
##      X firstname      sex  race call
## 4865 4865    Lakisha female black     0
## 4866 4866     Tamika female black     0
## 4867 4867     Ebony female black     0
## 4868 4868         Jay   male white     0
## 4869 4869    Latonya female black     0
```

Data Frames: using \$

The \$ operator is another way to access variables from a data frame:

```
head(resume$firstname, 3)
```

```
## [1] "Allison" "Kristen" "Lakisha"
```

Note: the “3” after the comma specifies how many observations to display.

Data Frames: using \$ (cont)

A variable in a data frame is like a vector, and we can apply functions to it

```
#Calculate mean call back rate  
mean(resume$call)
```

```
## [1] 0.08049281
```

Data Frames: using \$ (cont)

We can use the table function to describe character vectors

```
#Calculate number of stereotypically black and white names  
table(resume$race)
```

```
##  
## black white  
## 2435 2435
```

```
#Calculate number of stereotypically female and male names  
table(resume$sex)
```

```
##  
## female male  
## 3746 1124
```

Saving Objects

When you quit RStudio, you will be asked whether you would like to save the workspace. You should answer *no* to this in general: we only want to save what we want!

► To export CSV:

```
write.csv(resume, file = "resume.csv")
```

► To export *RData*:

```
save(resume, file = "resume.RData")
```

Go ahead and export your data frame as RData.